

Aashan — Repository Reconnaissance

Part 0 re-audit · AA provider evaluation · Part 3 readiness

2 September 2026 · `/Users/harieee/Documents/aashan_prooj`

Nothing in the repository was modified. Behaviour claims below were verified by running the code against a throwaway database outside the project folder, not inferred from reading it. The test suite was executed on a copy: **36 of 38 tests pass**; `tests/test_dashboard.py` (2 tests) could not be collected in the reconnaissance environment because Streamlit/Plotly were not installed there, which is an environment gap, not a failure.

Three things in the brief turned out not to match the repository, and the report follows the repository:

- The brief says ambiguous transactions become reviewable. In the running code, **no production path ever creates a reviewable candidate**. Everything is auto-confirmed.
- The brief treats aggregation as working. It computes **per-import totals, not per-user totals**, so a second import silently erases the first from the dashboard.
- The brief frames the AA decision as choosing a better provider. The binding constraint is **not the provider** — it is that only RBI/SEBI/IRDAI/PFRDA-regulated entities may be FIUs at all. Switching from Setu to Finvu does not change that, and on the evidence would increase friction rather than reduce it.

A. Current architecture summary

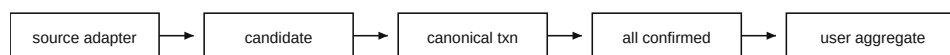
A FastAPI backend (~2,900 lines of application Python), two independent frontends, a repository boundary over SQLite and PostgreSQL, and versioned migrations for both dialects. The structure is sound and the layering is real — services do not reach into HTTP, adapters do not reach into aggregation, ownership is a dependency rather than a body field. The defects below are behavioural, not structural.

Area	State	Detail
Repository structure	Present	<code>app/{api, services, models, database}</code> , <code>migrations/{sqlite, postgres}</code> , <code>frontend/</code> , <code>dashboard/</code> , <code>tests/</code> , <code>reference/manus/</code> , <code>consumables/</code> (5 logo assets).
Backend architecture	Sound	FastAPI, 7 routers, service layer, Pydantic models, repository protocol with two implementations. No ORM models; hand-written SQL on both sides.
Frontend architecture	Incomplete	Two parallel UIs: vanilla-JS SPA (<code>frontend/</code> , 504 lines) and a Streamlit dashboard (<code>dashboard/</code> , 528 lines). Neither authenticates . See F9.
Schema & migrations	Present	17 tables. Postgres: 2 migrations, UUID keys, <code>NUMERIC(18,2)</code> , <code>timestampz</code> , <code>CHECK</code> constraints. SQLite: 4 migrations plus a compatibility shim that upgrades pre-Part-1 databases in place.
Auth	Backend done	Supabase JWT via PyJWT, HS256 secret or RS256 JWKS, issuer/audience/exp enforced, ownership from <code>sub</code> . Body <code>user_id</code> is never accepted. Service-role key never returned by <code>/api/auth/config</code> (tested).
RLS	Declared, unverified	17 tables <code>ENABLE ROW LEVEL SECURITY</code> + owner policies on <code>auth.uid()</code> . Not <code>FORCE d</code> , and no test exercises Postgres. See F5.
Transaction model	Good	Lifecycle, classification, budget, transfer and duplicate state are genuinely separate columns, not collapsed. Provenance columns present: <code>source</code> , <code>source_record_id</code> , <code>external_id</code> , <code>import_id</code> , <code>account_id</code> .
Candidate / review	Present but unreachable	Table, APIs and <code>review.py</code> all exist and work when driven directly. No ingestion path produces a candidate needing review. See F3.
Import / job model	Present	<code>imports</code> + <code>processing_jobs</code> with named checkpoints (<code>SOURCE_VALIDATED</code> → <code>NORMALIZATION_COMPLETED</code> → <code>CLASSIFICATION_*</code> → <code>PERSISTENCE_COMPLETED</code> → <code>AGGREGATION_COMPLETED</code>), error category and message. Executed synchronously, not by a worker.
AA abstraction	Partial	<code>AAProvider</code> ABC with four methods. Shape is provider-neutral; surrounding config, routing, webhook and schema are not. See section C.

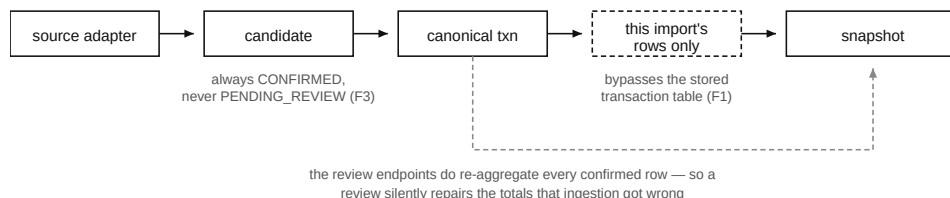
Setu-specific code	Widespread	Six files plus schema constraints and stored provider strings. Enumerated in section C.
CSV adapter	Working	Header-alias tolerant, extracts a strong reference for dedupe when the file has one, raises typed errors per row.
Aggregation	Broken	Deterministic and testable in isolation, but wired to the wrong input set. See F1.
Merchant rules	Minimal	Owner-scoped substring rules applied after the global classifier; user rule wins. No merchant identity entity — merchant is a free-text candidate on the transaction.
Classification	Working	Merchant rule (0.99) → keyword rule (0.95) → optional MiniLM embedding similarity → fallback /Other. Confidence and method are recorded per transaction.
Privacy / security	Mixed	Strong output redaction in <code>aa_client.py</code> ; that discipline is not applied in <code>webhooks.py</code> (F7). The <code>/api/dashboard/privacy</code> claim is no longer true (F8).
Test coverage	Real, narrow	38 tests: ownership isolation, transfer exclusion, budget exclusion, external-ID dedupe, checkpoints, review audit, Setu webhook idempotency. Zero tests on Postgres, zero negative security tests on the webhook, zero on repeat-import correctness.
PDF readiness	None	No PDF dependency in <code>requirements.txt</code> , no parser, no worker, no quarantine. 'PDF' exists only as a schema enum value and a type literal.
SMS readiness	Legacy only	<code>parse_sms</code> + POST <code>/api/ingest/sms</code> bypass the unified pipeline entirely and call <code>process_raw_rows</code> directly — no import, no candidate, no canonical transaction.
Provider replacement readiness	Partial	Interface reusable; identifiers, config, webhook and dedupe namespace are not. See section C.

Intended pipeline versus what actually runs

INTENDED



ACTUAL



Two different definitions of “the aggregate” coexist. Ingestion writes a snapshot of one import; review writes a snapshot of the whole account. Whichever ran last is what the dashboard shows.

B. AA provider recommendation

The constraint that decides this

Sahamati, which operates the ecosystem’s central registry, states the eligibility rule plainly:

“Currently, only players who are registered and regulated by either of the four Financial Service Regulators (FSR) — RBI, SEBI, IRDAI, PFRDA — are allowed to be FIPs and FIUs.”

Setu’s own go-live documentation confirms the same gate from the vendor side: the FIU submits its licence to Setu, who forwards it to Sahamati to establish eligibility. OneMoney’s onboarding documentation repeats it. Using a technology service provider does not remove it — Sahamati notes only that *the TSP itself* needs no NBFC-AA licence, which says nothing about the FIU.

So: a personal-finance application that is not an RBI/SEBI/IRDAI/PFRDA-regulated entity cannot become an FIU, on any provider. Every sandbox on the network is the front end of an onboarding funnel that terminates at that check. **The Setu difficulty is not a Setu problem, and no provider swap solves it.** The realistic production routes are (a) partnering with a regulated entity that acts as the FIU, or (b) obtaining a licence — reported as 5–10 months and ₹5–25 lakh in first-year cost, with mandatory FIP participation and recurring certification.

What a provider choice *can* still buy is items 1–11 of the sandbox checklist: a working consent journey, callbacks, a data session, an FI fetch, and a realistic ReBIT-shaped payload to normalise. That is worth having, and it is what the evaluation below optimises for.

Evaluation

Criterion	OneMoney / FinPro	Finvu (Cookiejar)	Anumati (Perfios)	Setu (incumbent)
Type	RBI-licensed AA	RBI-licensed AA	RBI-licensed AA	TSP, not an AA
Self-service signup	Yes — developer portal issues a test FIU ID and API key on basic company/app details	No — email <code>support@cookiejar.co.in</code> with root endpoint and public key	No self-service path found; sandbox page is request-based	Partial; docs public, credentials via sales
Sandbox availability	Yes, but a daily window (07:00–01:00 IST)	Yes, <code>aauat.finvu.in</code> ; tokens expire after 6 months	Advertised; not independently verifiable	Yes, <code>fiu-sandbox.setu.co</code>
Test accounts / FIP simulator	Configurable test accounts to “simulate various types of financial information”	Simulated FIP with a test-data entry API , ReBIT-conformant schemas, handles such as <code>customer1@finvu</code>	Not documented publicly	Sandbox FIP available
Consent journey	Android SDK now; web SDK “coming soon” , iOS “coming soon”	Redirect journey documented	Not documented publicly	Hosted redirect journey (already integrated here)
Callbacks / webhooks	Not documented on the pages reviewed	Three notification endpoints: consent, FI, account-link	Not documented publicly	Single notification endpoint (already integrated here)
Auth friction	API key	Detached JWS signing per request, public-key registration, publicly reachable callback root	Unknown	Client ID/secret + product instance header
Documentation	Portal docs, Postman collections	Best public ReBIT surface ; endpoints fully enumerated	Thin / gated	Strong product docs
Manual onboarding for production	Business agreement + Sahamati certification + registry + licence	Same	Same	Same, brokered by Setu

PRIMARY — OneMoney / MoneyOne FinPro. It is the only candidate where credentials are issued without a human in the loop, and it lets you configure test accounts. That gives the fastest feedback loop for checklist items 1–9. Accept two costs going in: the sandbox is unavailable roughly 01:00–07:00 IST, and the web SDK has not shipped, so a web-first consent journey may need the redirect flow or a mobile shell.

BACKUP — Finvu. Best public ReBIT documentation on the network and a genuine FIP simulator with a data-entry API, which is exactly what you need to generate realistic statements for normalisation. Onboarding is email-based, requests need detached JWS signatures, and you must expose a public callback root — the docs themselves concede the process “will be automated” in future, i.e. it is manual today. Slower to start, better to learn the specification against.

NOT RECOMMENDED as primary — Anumati. No verifiable self-service developer path or public API documentation was found. Selecting it would be a marketing-led choice, which the brief rules out.

Keep Setu. Do not delete the adapter. It is your only working end-to-end reference implementation and the only one that hands you decrypted JSON.

The recommendation the brief did not ask for

Because none of these can be tested to completion without a licence, **do not put AA on the critical path.** Two concrete consequences:

1. **Build a local ReBIT simulator** — a small FastAPI service implementing the consent → FI-request → FI-fetch contract, returning ReBIT-shaped deposit payloads, optionally encrypted. It costs a few hundred lines, runs in CI, is deterministic, and is the only way to test the AA adapter interface and the decryption path without a vendor. It also becomes the fixture generator for Parts 4, 5 and 7.
2. **Register on the SahamatiNet developer portal** as a parallel, low-cost track. It is an explicit proof-of-concept router with semi-self-service onboarding (form or email, then Sahamati validates). It is not yet a reliable feedback loop, but it is the ecosystem’s own direction of travel and cheap to hold a position in.

C. What must change to replace Setu

Keep — the abstraction is genuinely provider-neutral

`AAProvider` in `app/services/aa_client.py` already declares `create_consent`, `get_consent_status`, `create_data_session` and `fetch_data` in provider-free terms. The redaction helpers and the HTTP error taxonomy are reusable once renamed. Keep all of it.

Missing from the interface

The brief's target interface has three methods the code does not: `handle_callback`, `handle_webhook` and `normalize_response`. All three currently live in the HTTP layer as Setu-shaped logic, which is precisely why the router and the webhook are not portable. A fourth is missing from the brief and matters more than any of them — see the crypto note below.

Setu-specific code, by file

Location	Provider-specific	Generic — extract and keep
<code>app/config.py</code>	Eight <code>setu_*</code> settings; no provider selector exists	Needs <code>aa_provider</code> plus a per-provider credential namespace
<code>app/services/aa_client.py</code>	Endpoint paths <code>/v2/consents</code> , <code>/sessions</code> ; login at <code>accountservice.setu.co/v1/users/login</code> with <code>client: bridge</code> ; <code>x-product-instance-id</code> ; <code>build_consent_payload</code> (Setu's consent JSON); <code>SetuConfigurationError</code>	Redaction (<code>safe_*_text/url/error_details</code>), error classification taxonomy, bearer-token caching with expiry margin
<code>app/api/aa.py</code>	Entire router: <code>_real_provider</code> , <code>_setu_call</code> , callback params <code>id</code> / <code>success</code> / <code>errorcode</code> / <code>errmsg</code> , route naming	Consent request model, owner checks, consent-context persistence
<code>app/api/webhooks.py</code>	Route <code>/api/webhooks/setu</code> ; event names <code>CONSENT_STATUS_UPDATE</code> / <code>SESSION_STATUS_UPDATE</code> / <code>FI_DATA_READY</code> ; payload shapes for status, session ID and <code>notificationId</code>	Idempotency keying, session-processed guard, owner resolution from consent, ingestion dispatch
<code>app/services/setu_mapper.py</code>	Outer envelopes <code>fips[].accounts[].data.account</code> and <code>fiData[].data[].decryptedFI</code>	The inner account/transaction shape is ReBIT-standard and reusable across every AA
<code>app/services/ingestion/setu_adapter.py</code>	<code>source_type = "SETU"</code>	Extraction loop is otherwise generic

The expensive part is the identifier, not the code

"SETU" is not just a label. It is a value in the `SourceType` literal, a `CHECK` constraint on `imports.source` and `account_connections.source`, a hardcoded `provider = 'SETU'` in four repository methods, and — critically — **one third of the duplicate-detection key**: `create_transaction` deduplicates on `(user_id, source, external_id)`. Introduce a provider under a new source type and every previously imported transaction re-imports as new.

Recommendation: make the canonical source type "AA" and carry provider identity in the `provider` column that `consents` and `account_connections` already have. Accept 'SETU' as a legacy value with a backfill. Provider identity should never be a source type — otherwise every new AA needs a schema migration and forks the dedupe namespace.

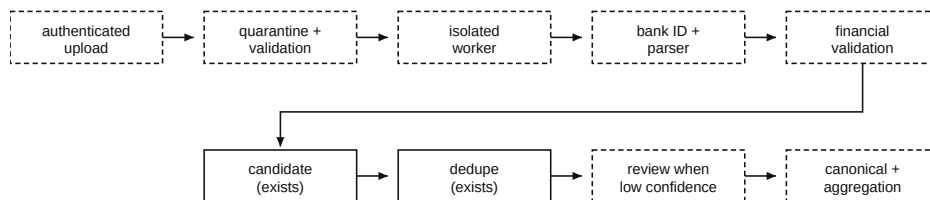
The hidden cost: there is no ReBIT crypto anywhere

Setu's Bridge returns decrypted JSON, so nothing in the repository performs ECDH key exchange, AES-GCM decryption, or detached JWS request signing. A direct AA integration — Finvu certainly, OneMoney depending on the path — requires all three, plus key generation, registration and rotation. The current `AAProvider` interface has no place to put any of it. **This is the largest unpriced item in the Setu replacement, and it argues for keeping a TSP-shaped adapter alongside any direct one.**

Sequence

1. Rename the generic surface off `setu_*`; add `AA_PROVIDER` config and a provider registry.
2. Move callback parsing, webhook parsing and response normalisation behind the interface as `handle_callback`, `handle_webhook`, `normalize_response`. Route becomes `/api/webhooks/aa/{provider}`.
3. Add crypto as an explicit, optional provider capability.
4. Migrate `SETU` → source `AA` + provider `SETU`, with a data backfill and a dedupe-continuity test.
5. Build the local ReBIT simulator; make it the provider-contract test fixture.
6. Implement OneMoney behind the same interface; run the full 14-step checklist.
7. Only then consider retiring the Setu adapter — and probably don't.

D. Part 3 readiness assessment



----- does not exist in the repository today

——— exists — but see F1, F2, F3 before relying on it

Nine of the thirteen stages are absent; of the four that exist, three are defective in ways PDF ingestion would amplify.

Present and reusable

The import/job/checkpoint machinery, the candidate table, the canonical transaction model with separate state dimensions, the review APIs, the adapter ABC, and ownership scoping. That is a real foundation; Part 3 does not need to invent it.

Absent

- **Any PDF capability at all.** No PDF library in `requirements.txt`. No parser, no bank identification, no confidence model.
- **Any isolation.** There is no worker, no queue, no subprocess boundary, no resource limit, no quarantine, no temp-file policy, no cleanup. Ingestion runs inline in the API process.
- **Any upload bound.** `await file.read()` pulls the entire body into memory with no size cap — already a denial-of-service surface for CSV, and a decompression-bomb surface for PDF.
- **Extraction-stage fields.** `NormalizedTransactionInput` has no balance, no opening/closing balance, no statement period, no account identifier, no extraction confidence. `imports` has a `page_count` column that nothing writes, and no columns for identified bank, parser used, or parser confidence. A migration is required before the first parser.
- **Financial validation.** Nothing validates balance continuity, ordering, opening/closing consistency, or debit/credit arithmetic — because until now no source produced numbers that needed checking.

Four preconditions, in order

Each of these is amplified rather than merely inherited by PDF ingestion:

1. **F1 — aggregation.** A user uploading three monthly statements would see only the third month's totals. Aggregation must derive from the stored confirmed transactions, as `review.py` already does.
2. **F2 — deduplication.** Bank statement PDFs carry no external transaction ID, and statement periods overlap. Without a content-based key (owner + account + date + amount + normalised description) every re-upload double-counts. This is the single most likely way Part 3 produces wrong numbers.
3. **F3 — the review path.** Low-confidence extraction “must become reviewable”. There is no reachable code path that makes anything reviewable. This must be built before, not after, a parser that will routinely be uncertain.
4. **Asynchronous processing.** `ingest_with_adapter` is synchronous and returns the aggregate in the HTTP response. That contract cannot survive a sandboxed parser worker, and it is already wrong: `POST /api/ingest/csv` is an `async def` that calls blocking SQLite and, when embeddings are enabled, a transformer model — directly on the event loop.

E. Security concerns

ID	Severity	Finding	Detail and evidence
F4	CRITICAL	The Setu webhook is unauthenticated and unsigned	<code>app/api/webhooks.py</code> · <code>_validate_notification</code> checks only that <code>type</code> is in a three-item set and that <code>consentId</code> is present. A code comment states no signature scheme is implemented. With <code>MOCK_MODE=false</code> and a public URL, anyone who learns or guesses a consent ID can POST <code>FI_DATA_READY</code> with an arbitrary FI payload and have canonical transactions written to that owner's account, and can drive <code>update_aa_consent_status</code> . Replay protection keys on <code>notificationId</code> , which is attacker-supplied — vary it and the guard is void. Fix before <code>MOCK_MODE=false</code> is ever exposed.
F5	CRITICAL	PostgreSQL RLS is very likely not enforced for the application	<code>migrations/postgres/001</code> uses <code>ENABLE ROW LEVEL SECURITY</code> , never <code>FORCE</code> . The application role runs the migrations and therefore owns the tables, and a table owner is exempt from its own RLS unless <code>FORCE</code> is set . If <code>DATABASE_URL</code> is a Supabase <code>postgres</code> or service-role connection, RLS is bypassed outright and <code>_set_user_context</code> is decorative. Policies also depend on <code>auth.uid()</code> , which exists only on Supabase — the migration fails on plain PostgreSQL. No test touches PostgreSQL, so the whole layer is unverified. Add <code>FORCE ROW LEVEL SECURITY</code> , connect as a non-owning least-privilege role, and add a two-user cross-tenant negative test against a real Postgres.
F6	MEDIUM	Cross-user read of webhook audit rows	Policy <code>audit_events_owner</code> is <code>USING (user_id IS NULL OR user_id = auth.uid())</code> , and <code>mark_aa_webhook_event</code> inserts with <code>user_id NULL</code> . Every authenticated user can therefore read every webhook audit row, including other users' consent and session identifiers. Attribute the row to its owner, or move these to a table with no user-facing policy.
F7	MEDIUM	Raw provider response bodies leak to API callers	<code>webhooks.py</code> · <code>_fetch_setu_data</code> and <code>_create_setu_session</code> put <code>exc.response.text[:500]</code> into the <code>HTTPException</code> detail. This bypasses the careful <code>safe_setu_error_details</code> allow-list used everywhere in <code>aa_client.py</code> — an error body containing account or token material is returned verbatim. Route both through <code>describe_setu_http_error</code> .
F9	MEDIUM	Neither frontend authenticates	<code>frontend/app.js</code> never sets an <code>Authorization</code> header and has no Supabase client; <code>dashboard/api_client.py</code> uses bare <code>requests</code> . Since <code>ENVIRONMENT=production</code> forces <code>auth_required</code> , both UIs return 401 on every call the moment the system is deployed as intended . Part 1 is backend-only, and the roadmap defers this to Part 9.
F12	MEDIUM	No version control; live credentials on disk	There is no <code>.git</code> directory. A financial codebase with no history, no rollback, no diff and no review. Separately, <code>.env</code> holds non-empty <code>SETU_CLIENT_ID</code> , <code>SETU_CLIENT_SECRET</code> and <code>SETU_PRODUCT_INSTANCE_ID</code> . It is correctly gitignored — but there is no repository for it to be ignored from. Initialise git and rotate those credentials before any remote exists.
F20	MEDIUM	Blocking work on the async event loop	<code>POST /api/ingest/csv</code> is <code>async def</code> and calls the synchronous <code>ingest_with_adapter</code> — SQLite I/O, and a lazily-loaded MiniLM model that may download weights at request time. Several <code>app/api/aa.py</code> handlers do the same with sync repository calls. One slow import stalls the whole process.
F10	Low	Migrations run on every database call	<code>PostgresRepository.initialize()</code> calls <code>run_postgres_migrations()</code> , and every repository method calls <code>initialize()</code> first — a full migration scan and DDL transaction per API call, with the attendant lock risk. Run once at startup.
F11	Low	Postgres migration runner splits on semicolons	<code>connection.py</code> splits each file on <code>;</code> . Any future function body, DO block or trigger breaks silently mid-statement. The <code>-- STATEMENT BREAK --</code> markers in the SQL suggest an intent the runner does not implement.

F. Financial correctness concerns

F1 — Aggregation is per-import, not per-user (CRITICAL)

`ingest_with_adapter` collects only the rows of the current import into `categorized_rows` and hands them to `process_raw_rows`, which aggregates exactly what it was given and saves it as the newest snapshot. `latest_aggregate` then returns that snapshot as though it were the account total. Reproduced:

```
import 1 (SWIGGY ₹500, July) → dashboard total_spending = 500.0
import 2 (UBER ₹300, August) → dashboard total_spending = 300.0 ← July erased
                                monthly = {'2026-08': 300.0}
                                stored transactions = 2

then one unrelated review action → total_spending = 800.0
                                monthly = {'2026-07': 500.0, '2026-08': 300.0}
```

Two contradictory definitions of “the aggregate” coexist: ingestion snapshots one import, `review._reaggregate_confirmed` snapshots the whole account. Whichever ran last wins. Every downstream number — dashboard, budgets, anomalies, insights — inherits this. **Multi-source reconciliation, the stated purpose of Parts 3, 6 and 7, is impossible until aggregation derives from the canonical transaction table.** The fix is small: have ingestion call the same re-aggregation that review already uses.

F2 — Deduplication depends entirely on an external ID (CRITICAL)

`create_transaction` checks for an existing row only when `item.external_id` is set. The only other guard is a client-supplied Idempotency-Key header. With neither, the same file ingests cleanly twice. Reproduced:

```
same CSV, twice, no Idempotency-Key, no reference column
→ rows_confirmed = 1, 1     duplicates = 0
→ stored transactions = 2
```

The existing test, `test_strong_external_id_duplicate_is_ignored`, verifies the case that works and there is no test for the case that does not. Bank statement PDFs have no external transaction ID and overlapping periods are routine, so Part 3 will double-count on the first re-upload. A content-derived key over owner, account, date, signed amount and normalised description is needed, with near-matches becoming `POSSIBLE_DUPLICATE` rather than being silently dropped.

F3 — Nothing is ever reviewable (HIGH)

`ingestion/service.py` passes `review_status="CONFIRMED"` and `transaction_status="CONFIRMED"` unconditionally, including when it has just computed `classification_status="AMBIGUOUS"`. Reproduced with a person-name description — exactly the case the brief calls out:

```
"RAHUL KUMAR UPI" → classification_status = AMBIGUOUS
                   transaction_status    = CONFIRMED
                   candidates pending review = 0
```

Grepping the application for `PENDING_REVIEW` outside the type literal returns nothing. The state exists in the model, the schema, the review service and the API — and is reachable only from test code. The product principle “ambiguous transactions must become reviewable, not silently misclassified” is currently not implemented.

F8 — The privacy claim is no longer true (HIGH, NON-TECHNICAL RISK)

`privacy_database_status` reports `raw_transactions_persisted: false` by checking for three table names — `raw_transactions`, `sms_messages`, `aa_payloads` — that have never existed. The check cannot fail. Its own comment says the `transactions` table “is currently empty”, which stopped being true in Part 2: full narration text is now persisted (verified: `'RAHUL KUMAR UPI'` stored verbatim). The SPA meanwhile renders “No raw descriptions — narrations and SMS text are not returned to the dashboard after processing”, and `GET /api/transactions` returns them. Persisting descriptions is a defensible architectural change; **advertising the opposite to users is not.** Align the claim with the storage, in both directions: make the check assert something falsifiable, and update the README and UI copy.

Lower-severity correctness

- **F13 — Time handling.** `_parse_date` returns naive datetimes; PostgreSQL columns are `timestampz`, so values are reinterpreted in the server timezone and month buckets (`%Y-%m`) can shift across boundaries. Nothing carries a locale hint, so an ambiguous PDF date like `03/04/2026` is resolved by format order alone.
- **F14 — Money is float.** `Transaction.amount` is `float` and SQLite stores `REAL`, while PostgreSQL uses `NUMERIC(18,2)`. Development and production can disagree on a rounded total, and `aggregate_transactions` accumulates in float. Use `Decimal` or integer paise before balance-continuity validation, which will otherwise fail on representation error.
- **F17 — Credits are marked budget-INCLUDED.** `service.py` sets `budget_status` from transfer state alone, so income rows are `INCLUDED`. Harmless today because the aggregator skips credits before reading it — but Part 5 will read it.
- **F18 — Reflection-based kernel coupling.** `_pipeline_call` inspects the processor's signature to decide whether to pass `user_id`, `source` and `import_id`. A signature change silently drops ownership rather than failing. Too clever for the financial kernel.
- **F19 — Mock AA state is process-global.** `mock_provider` is instantiated at import time and keeps consents in a module dict. Fine for a demo; wrong under multiple workers.

G. Risks in the existing roadmap

1. **Part 2 is reported complete but the pipeline it establishes computes the wrong totals.** Everything from Part 3 onward is layered on aggregation that loses data on the second import. The tests pass because each one ingests once. Add a repeat-import regression test — it is a four-line test that would have caught this.
2. **Part 3 is sequenced before the correctness work it depends on.** PDF is the hardest, least trustworthy source, and it lands on a pipeline with no content-based dedupe and no review path. Split it: *Part 3a* — aggregate from stored transactions, content-hash dedupe, route low-confidence rows to review, move ingestion to a worker; *Part 3b* — parsers. 3a is also a prerequisite for Parts 6 and 7 and pays for itself three times.
3. **Part 9 (frontend) is too late.** Turning on `AUTH_REQUIRED` breaks both UIs completely (F9), so the system cannot be run in its intended production configuration until Part 9. That means Parts 3–8 will all be validated in an authentication mode that production never uses, and every ownership bug hides until the end. Wire Supabase into one frontend now; retire the other.
4. **Part 7 (AA) may never complete, and the roadmap has no branch for that.** Without a regulated-entity licence or partner there is no production AA. Restate Part 7 as “provider-pluggable and verified against a local ReBIT simulator”, and make the licence question a business track running in parallel rather than an engineering dependency.
5. **Two frontends is a duplicate system the engineering rules forbid.** The Streamlit dashboard and the vanilla SPA render overlapping views from the same endpoints, and neither authenticates. Pick one.
6. **Part 6 (SMS) inherits a bypass.** `POST /api/ingest/sms` calls `process_raw_rows` directly — no import, no candidate, no canonical transaction, no dedupe. It is the counter-example to “every ingestion source converges into one pipeline”, and it is already in production code. Converge it during 3a while the seam is open.
7. **Part 8 (AI) currently has nothing to consume.** The stated privacy design is that AI receives sanitised aggregates — but the aggregate is wrong (F1) and the transaction table now holds raw narrations (F8). Settle what the sanitised layer actually is before building on it.
8. **No dependency pinning discipline for a security-sensitive parser.** `requirements.txt` pins exactly, which is right, but it also carries `torch`, `transformers` and `sentence-transformers` — roughly 2 GB of attack surface — into the same image that will soon parse hostile PDFs. Split the classifier into its own service or make embeddings an explicit optional extra before Part 3.

H. Verdict

GO WITH CHANGES — and the changes are prerequisites, not parallel work.

The architecture is worth building on. The layering is honest, ownership is derived correctly from the JWT, state dimensions are properly separated, provenance is preserved, and the adapter boundary is real. None of that needs rewriting.

But three defects in the pipeline Part 3 would sit on top of are the kind that produce quietly wrong money: totals that lose an import, duplicates that pass unnoticed, and an ambiguity signal that is computed and then discarded. A PDF parser makes all three worse rather than exposing them.

Before the first parser: F1 (aggregate from stored transactions), F2 (content-based deduplication), F3 (route low-confidence rows to review), and move ingestion off the request thread onto a worker. Each is small; together they are Part 3a.

Before `MOCK_MODE=false` meets a public URL: F4 (authenticate the webhook).

Before any real user data reaches PostgreSQL: F5 (force RLS, connect as a non-owning role, and test it).

Today, unconditionally: initialise git and rotate the credentials sitting in `.env`.

On the AA question the answer is *not* a provider swap. OneMoney is the best sandbox and Finvu the best documentation, but neither can be tested to completion without a licence Aashan does not hold. Keep the abstraction, keep Setu, build a local ReBIT simulator, and take AA off the critical path.

Sources

- Sahamati — [How to join the Account Aggregator Network](#) (FIU eligibility, certification, central registry)
- Sahamati — [Account Aggregator Sandbox](#) (list of AA sandboxes)
- Sahamati — [Technology Service Providers](#) (TSP licensing)

- SahamatiNet — [Sandbox Onboarding](#) (POC router, semi-self-service)
- Finvu — [Sandbox overview](#) and [AA integration](#) (email onboarding, JWS, endpoints, token expiry)
- MoneyOne — [FinPro technical docs](#) and [onboarding procedure](#) (self-service portal, sandbox hours, SDK status)
- Setu — [FIU go-live process](#) (licence forwarded to Sahamati)
- CASParser — [State of Account Aggregator in 2026](#) (timeline and cost estimates)